

CLASE 07 — TU PROYECTO ASIGNADO

DML y consultas SQL básicas aplicadas a los datos reales de tu dominio

Guía de trabajo del estudiante · Observa ParkFlow 360, replica el método en TU proyecto

REGLA CENTRAL: ParkFlow 360 es el ejemplo del profesor. Tu evidencia se construye exclusivamente sobre el proyecto asignado a tu equipo. Debes copiar el MÉTODO, no las tablas, nombres ni datos de ParkFlow.

1. Qué debes conseguir en esta clase

En la Clase 06 tu equipo creó físicamente las primeras tablas. Hoy esas tablas deben comenzar a contener datos coherentes y a responder preguntas reales del dominio. El objetivo no es llenar tablas “porque sí”, sino aprender a insertar, consultar, filtrar, ordenar y modificar información con intención.

Al finalizar	Debes poder demostrar
Datos	Un dataset mínimo coherente en tus tablas núcleo
Lectura	SELECT con columnas explícitas
Filtros	WHERE con varias condiciones útiles
NULL	IS NULL / IS NOT NULL cuando corresponda
Cambios	UPDATE seguro con verificación
Eliminación	Decisión justificada: DELETE o conservación/inactivación
Git	Scripts y consultas versionados

2. Qué debes observar mientras el profesor trabaja ParkFlow 360

- Cómo decide qué datos de ejemplo son coherentes entre tablas relacionadas.
- Cómo respeta el orden padre → hijo al insertar.
- Cómo predice qué filas devolverá un SELECT antes de ejecutarlo.
- Cómo utiliza WHERE para expresar una pregunta del negocio.
- Cómo maneja NULL.
- Cómo verifica un UPDATE antes y después.
- Por qué no aplica DELETE indiscriminadamente.
- Cómo convierte la consola SQL en archivos versionados y reproducibles.

3. Lo que NO debes copiar

- No copies parking_lot, zone, parking_space, customer ni otros nombres de ParkFlow salvo que realmente existan en tu dominio.
- No copies los datos de estacionamientos como dataset de tu proyecto.
- No copies consultas sólo para “cumplir cantidad”; formula preguntas útiles para tu ficha oficial.
- No inventes DELETE si tus reglas exigen conservar históricos.

4. Archivos que debes tener abiertos

- ☐ Ficha oficial del proyecto asignado.
- ☐ modelo-conceptual-v0.1.md.
- ☐ model-relational-v0.1.md.
- ☐ DER lógico/diccionario de Clase 04.
- ☐ modelo-fisico-v0.1.md y plan-migracion-v1.md.
- ☐ V1_init_core.sql de la Clase 06.
- ☐ Repositorio actualizado y base de datos local conectada desde DataGrip.

5. Paso 1 — Diseña tu dataset antes de insertarlo

Elige 4–6 tablas núcleo ya creadas. Para cada tabla, define entre 2 y 5 filas que permitan demostrar relaciones, filtros y estados de tu dominio.

Pregunta	Ejemplo de respuesta de TU proyecto
¿Qué tabla padre necesita datos primero?	Categoría, cliente, usuario, sede, profesional, etc.
¿Qué tabla hija depende de ella?	Producto, vehículo, ejemplar, cita, reserva, envío...
¿Qué estados necesitas representar?	Al menos dos estados válidos si la tabla los maneja.
¿Dónde conviene un NULL legítimo?	Sólo donde tu modelo lo permita.
¿Qué dato debe ser único?	Código, placa, ISBN/ejemplar, tracking, etc. según proyecto.

Tu dataset debe ayudarte a PROBAR el modelo. Si todas las filas son iguales, no podrás practicar filtros útiles.

6. Paso 2 — Crea un script reproducible de datos

Crea, de acuerdo con la convención de tu repositorio:

```
backend/src/main/resources/db/migration/V2__seed_core.sql
```

Plantilla genérica:

```
INSERT INTO tabla_padre (codigo, nombre)
```

VALUES

```
( 'P01', 'Dato 1'),
( 'P02', 'Dato 2');
```

```
INSERT INTO tabla_hija (tabla_padre_id, codigo, estado)
```

VALUES

```
(1, 'H01', 'ACTIVO'),
(1, 'H02', 'ACTIVO'),
(2, 'H03', 'INACTIVO');
```

Adapta columnas, estados y valores a TU modelo. No ejecutes esta plantilla literalmente si esas tablas no existen.

7. Paso 3 — Predice antes de ejecutar

1. Lee el INSERT completo.
2. Comprueba que cada FK tenga un padre válido.
3. Comprueba UNIQUE, NOT NULL y CHECK.
4. Predice cuántas filas debería insertar.
5. Ejecuta.
6. Verifica el número de filas afectadas y consulta los datos.

8. Paso 4 — Aprende a consultar con SELECT

```
SELECT columna_1, columna_2, columna_3
FROM tu_tabla
ORDER BY columna_2;
```

No dejes SELECT * como única evidencia. Debes poder justificar por qué cada columna aparece en el resultado.

9. Paso 5 — Formula preguntas reales de TU dominio

Debes construir al menos 8 consultas. Úsalas para contestar preguntas de negocio de tu ficha, por ejemplo:

Dominio	Ejemplo de pregunta, NO de SQL
Biblioteca	¿Qué ejemplares están disponibles?
Farmacia	¿Qué productos/lotas están activos o próximos a una condición relevante?
Taller	¿Qué órdenes están en un estado específico?
Paquetería	¿Qué envíos están pendientes o en reparto?
Salón/SPA	¿Qué servicios/profesionales están activos?

Alquiler	¿Qué unidades están disponibles o bloqueadas?
Tu proyecto	Formula preguntas equivalentes usando tus actores, estados y reglas.

10. Herramientas SQL mínimas de hoy

Herramienta	Uso
WHERE	Filtrar filas
=, <>, >, >=, <, <=	Comparar valores
AND / OR / NOT	Combinar condiciones
IN	Aceptar uno de varios valores
BETWEEN	Filtrar un rango inclusivo
LIKE / ILIKE	Buscar patrones de texto
IS NULL / IS NOT NULL	Comprobar ausencia/presencia
ORDER BY	Ordenar explícitamente
LIMIT	Limitar filas devueltas

11. Paso 6 — Practica NULL sólo si existe en tu modelo

```
SELECT id, columna_opcional
FROM tu_tabla
WHERE columna_opcional IS NULL;
```

```
SELECT id, columna_opcional
FROM tu_tabla
WHERE columna_opcional IS NOT NULL;
```

No escribas columna = NULL. NULL no se trata como un valor común. Y no inventes columnas opcionales sólo para poder hacer este ejercicio.

12. Paso 7 — UPDATE: usa la regla SELECT → UPDATE → SELECT

```
-- 1. Identifica la fila
SELECT id, estado
```

```
FROM tu_tabla
WHERE id = 1;

-- 2. Modifica con condición
UPDATE tu_tabla
SET estado = 'NUEVO_ESTADO_VALIDO'
WHERE id = 1;

-- 3. Verifica
SELECT id, estado
FROM tu_tabla
WHERE id = 1;
```

Nunca pruebes un UPDATE importante escribiéndolo directamente sin saber qué filas afectará. Un UPDATE sin WHERE puede modificar toda la tabla.

13. Paso 8 — Decide si DELETE tiene sentido en TU dominio

Revisa las reglas de negocio de tu ficha. Si existen históricos, auditoría o trazabilidad, probablemente varias filas no deban eliminarse físicamente.

Situación	Decisión que debes justificar
Catálogo corregible sin historial	DELETE podría ser válido en casos controlados.
Orden/venta/préstamo/reserva histórica	Normalmente conservar y cambiar estado.
Entidad referenciada por otras	FK puede impedir borrado; analiza por qué.
Registro temporal de laboratorio	Puede eliminarse para practicar, si no rompe relaciones.

14. Paso 9 — Crea tu archivo de consultas de estudio

```
docs/05-database/clase07-consultas.sql
```

Estructura recomendada:

```
-- Q01. [Pregunta de negocio]
SELECT ...
FROM ...
WHERE ...
ORDER BY ...;

-- Q02. [Pregunta de negocio]
SELECT ...;
```

```
-- Q03. [Pregunta de negocio]
SELECT ...;
```

15. Las 8 consultas mínimas de tu equipo

- ☐ 1 consulta de listado ordenado.
- ☐ 2 consultas con filtros simples.
- ☐ 1 consulta con AND u OR.
- ☐ 1 consulta con IN o BETWEEN cuando sea coherente.
- ☐ 1 consulta con LIKE/ILIKE sobre un campo textual útil.
- ☐ 1 consulta con IS NULL/IS NOT NULL si tu diseño contiene un campo opcional.
- ☐ 1 consulta adicional directamente vinculada a una regla o necesidad de tu dominio.

Puedes superar ocho consultas. La calidad se mide por intención y comprensión, no por volumen.

16. Paso 10 — Provoca al menos un error y entiéndelo

- Intenta insertar un duplicado que viole un UNIQUE, o una FK inexistente, o un estado que viole CHECK.
- Lee el mensaje de PostgreSQL.
- Identifica qué constraint lo rechazó.
- Explica por qué ese error protege el dominio.
- No “arregles” el constraint sólo para que pase un dato inválido.

17. Revisión cruzada obligatoria

- ☐ Otro integrante revisa el V2__seed_core.sql.
- ☐ Otro integrante ejecuta al menos 3 consultas sin que el autor se las explique previamente.
- ☐ Cada integrante puede explicar una FK involucrada en los INSERT.
- ☐ Cada integrante puede explicar por qué una consulta devuelve exactamente esas filas.
- ☐ El equipo identifica qué entidades NO deben borrarse físicamente.

18. Evidencia Git

```
git status

git add backend/src/main/resources/db/migration/V2__seed_core.sql docs/05-database/clase07-consultas.sql

git commit -m "feat(db): add seed data and basic queries"

git log --oneline -5
```

19. Errores que debes evitar

Error	Qué demuestra
-------	---------------

Copiar datos/consultas de ParkFlow	No transferiste el aprendizaje a tu dominio.
SELECT * como única forma de consulta	No defines qué información necesitas.
= NULL	No comprendiste NULL en SQL.
UPDATE sin WHERE	Riesgo grave de modificación masiva.
DELETE de históricos	No relacionas SQL con reglas de negocio.
Consultas sin comentario/pregunta	No existe intención verificable.
Editar datos sólo desde la grilla	No practicas SQL defendible.

20. Preguntas que debes poder responder

- ¿Qué diferencia existe entre una tabla vacía y una tabla con dataset de prueba?
- ¿Por qué insertaste primero esa tabla y después la siguiente?
- ¿Qué constraint podría rechazar este INSERT?
- ¿Por qué usaste IS NULL y no = NULL?
- ¿Qué filas modifica exactamente tu UPDATE? ¿Cómo lo sabes?
- ¿Por qué una de tus entidades no debe eliminarse físicamente?
- Muestra una consulta de tu proyecto y explica cada cláusula de izquierda a derecha.

21. Evidencia mínima obligatoria

- ☐ Base con datos coherentes en 4–6 tablas núcleo.
- ☐ V2__seed_core.sql reproducible.
- ☐ Archivo clase07-consultas.sql.
- ☐ Al menos 8 consultas con intención de negocio.
- ☐ Un UPDATE seguro demostrado.
- ☐ Prueba de un constraint fallando correctamente.
- ☐ Decisión documentada sobre DELETE/históricos.
- ☐ Commit de cierre de clase.

22. Autoevaluación rápida

Nivel	Diagnóstico
VERDE	Puedo escribir y explicar INSERT/SELECT/WHERE/ORDER BY/UPDATE sin copiar.
AMARILLO	Puedo ejecutarlos, pero todavía necesito ejemplos para construir filtros.

ROJO	Sólo puedo modificar datos desde la grilla o copiar SQL sin predecir el resultado.
------	--

23. Definition of Done — Tu equipo

- ☐ El dataset representa situaciones reales de nuestro proyecto.
- ☐ Las FK de los datos son coherentes.
- ☐ Las consultas responden preguntas del dominio.
- ☐ Usamos ORDER BY cuando el orden importa.
- ☐ Entendemos NULL.
- ☐ No ejecutamos UPDATE sin condición deliberada.
- ☐ No borramos históricos sin justificación.
- ☐ Los scripts están en Git.
- ☐ Todos los integrantes pueden explicar al menos tres consultas.

24. Preparación para la Clase 08

Clase 08: JOIN. Hasta hoy consultaste principalmente una tabla a la vez. En la siguiente clase tendrás que responder preguntas que atraviesan relaciones: por ejemplo cliente → vehículo, libro → ejemplar, ruta → envío, profesional → cita o cualquier relación equivalente de TU proyecto.